

Diversidad textual y tokens efectivos: un estudio con modelos de lenguaje autoregresivos pequeños

Nicolás Meléndez

Universidad de los Andes, Bogotá
nicolasmelendez30@gmail.com

Mayo 2026

Resumen. Las leyes de escalamiento relacionan el desempeño de modelos de lenguaje con parámetros, datos y cómputo, pero el conteo bruto de tokens no describe por sí solo el valor informacional de un corpus. Este trabajo estudia, en un régimen pequeño y reproducible, si la diversidad textual empírica sirve como proxy de tokens efectivos en el sentido de Chang et al. Se implementa en el repositorio `nicolas-lm` un pipeline de modelado autoregresivo (tokenizador, dataset de predicción del siguiente token, métricas de corpus, Transformer causal y una variante de tipo LLaMA) y se formalizan los objetos matemáticos que ejecuta el código: red neuronal entrenable, modelo autoregresivo, riesgo poblacional y empírico, entropía cruzada y AdamW. Con dos corpus de 100 000 caracteres y el mismo presupuesto de entrenamiento, el corpus más diverso presenta mayor entropía de n -gramas, mayor distinct-4 y menor compresibilidad, pero obtiene mayor pérdida de test y mayor brecha de generalización en ambas arquitecturas. En este estudio, diversidad local no implica efectividad de tokens. [Estas notas resumen los resultados; un documento complementario, `mathematical\_background.tex` en el mismo repositorio, desarrolla los preliminares matemáticos en detalle.](#)

Palabras clave. tokens efectivos, Transformer, LLaMA-style, riesgo empírico, AdamW.

1 Introducción

El entrenamiento moderno de modelos de lenguaje se entiende en buena medida por un lente de leyes de escalamiento: al variar parámetros, datos y cómputo, la pérdida de test sigue tendencias empíricas relativamente estables [1, 2]. Estas leyes, sin embargo, no resuelven una cuestión estadística fundamental: dos corpus con el mismo número de tokens pueden inducir distribuciones de entrenamiento muy distintas. Chang et al. [3] formulan esta diferencia mediante la noción de *tokens efectivos*: el aporte informacional de un corpus depende de su calidad y composición, y no únicamente de su tamaño.

Definición 1 (Corpus y token). Un *corpus* es una sucesión finita $s = (c_0, \dots, c_{N-1})$ de símbolos sobre un alfabeto finito V . Un *tokenizador* es una aplicación $\tau : \bigcup_{n \in \mathbb{N}} V^n \rightarrow \bigcup_{m \in \mathbb{N}} T^m$, donde T es un alfabeto finito de tokens. En este trabajo se usa el caso particular del *nivel carácter*, en el que τ restringe a una biyección $V \rightarrow T$ aplicada símbolo por símbolo, de modo que N caracteres producen N tokens.

Definición 2 (Token efectivo, [3]). Dado un corpus s de tamaño D , Chang et al. definen

$$D_q = D \cdot \exp(c_1 \text{Dr}(s) + c_2 S(s)),$$

donde $\text{Dr}(s) = |s|/|\text{gzip}(s)|$ es el inverso de la razón de

compresión gzip^1 de s (mayor valor, mayor *diversidad*), y $S(s) = \text{PPL}_\theta(s)^{-1}$ es el inverso de la perplejidad de s bajo un *teacher model* p_θ . La perplejidad se define como $\text{PPL}_\theta(s) = \exp(-\frac{1}{N} \sum_{i=1}^N \log p_\theta(x_i | x_{<i}))$ y mide cuán predecible es el corpus para el modelo: si $\text{PPL}_\theta(s) = k$, el modelo se comporta, en promedio, como si en cada posición tuviera que elegir uniformemente entre k tokens. Valor bajo de PPL (equivalentemente, valor alto de S) corresponde a alta previsibilidad, lo que Chang et al. denominan *syntheticity*. Las constantes c_1, c_2 se ajustan empíricamente. Conceptualmente, el número de tokens efectivos es la cantidad útil de datos relativa a una arquitectura, un tokenizador y un presupuesto de cómputo.

Este trabajo aísla únicamente una parte medible de D_q : la diversidad empírica del texto, sin invocar modelos externos. La pregunta es

$$\text{¿} D(\text{High}) > D(\text{Medium}) \Rightarrow \hat{R}_{\text{test}}(\text{High}) < \hat{R}_{\text{test}}(\text{Medium}) \text{?} \quad (1)$$

En palabras: dados dos corpus con el mismo número de tokens y entrenando el mismo modelo durante el mismo número de pasos sobre cada uno, ¿el corpus con mayor diversidad textual (medida por las métricas $H_k, D_n, \rho_{\text{gzip}}$ que se definen en §10)

¹gzip denota el compresor sin pérdida estándar (LZ77 + Huffman); $|\text{gzip}(s)|$ es la longitud en bytes de s tras codificarlo en UTF-8 y comprimirlo.

produce un modelo que generaliza mejor a texto no visto? Explícitamente, *no* se mide syntheticity (perplejidad bajo un teacher model), factualidad ni coherencia semántica.

2 Tokenización

Definición 3 (Tokenizador a nivel carácter). Sea s un corpus. El módulo `tokenizer.py` construye el vocabulario

$$\mathcal{V} = \{c : c \in s\}, \quad \tau : \mathcal{V} \rightarrow \{0, \dots, |\mathcal{V}| - 1\},$$

donde τ es una biyección. La sucesión tokenizada es $x_t = \tau(c_t)$ para $t = 0, \dots, N - 1$. La clase `CharTokenizer` expone `encode`, `decode` y `vocab_size`, e implementa internamente los diccionarios `stoi` (*string-to-integer*, asocia cada carácter del vocabulario con su id entero) e `itos` (*integer-to-string*, la operación inversa).

A diferencia de tokenizadores subpalabra como BPE o SentencePiece [4, 5], el nivel carácter reduce realismo lingüístico pero garantiza que el espacio de medición del corpus (la cadena de símbolos) coincida con el espacio de predicción del modelo (la cadena de tokens). Esta elección simplifica el análisis sobre tokens efectivos: el conteo en caracteres del corpus coincide con el conteo en tokens vistos por el modelo, y las métricas de diversidad de §10, definidas sobre los caracteres del corpus, son directamente comparables con el presupuesto de entrenamiento, medido en tokens.

3 Dataset autoregresivo

Dado un contexto finito $B = 64$, el módulo `data.py` construye en `TokenDataset` ejemplos supervisados por desplazamiento de una posición:

$$X_i = (x_i, \dots, x_{i+B-1}), \quad Y_i = (x_{i+1}, \dots, x_{i+B}). \quad (2)$$

El parámetro B , llamado *block size* o longitud de contexto, determina cuántos tokens anteriores ve el modelo al predecir el siguiente. La elección $B = 64$ responde a un compromiso entre dos restricciones: B debe ser lo suficientemente grande para capturar dependencias locales informativas (palabras y frases cortas a nivel carácter ocupan del orden de varias decenas de caracteres) y lo suficientemente pequeño para que la atención causal, cuyo costo en memoria y cómputo escala como $O(B^2)$, siga siendo viable a la escala de este experimento.

La tarea es *predicción del siguiente token*: dado X_i , el modelo asigna una distribución de probabilidad sobre el vocabulario $\mathcal{V}$ para cada una de las B posiciones, y se compara con Y_i , que contiene los tokens correctos. Heurísticamente, si los últimos caracteres son "la casa es " (con espacio final), el modelo debería asignar más masa de probabilidad a letras que comienzan palabras españolas comunes ('g' de "grande", 'b' de "bonita", 'm' de "mía") que a letras improbables en esa posición ('x', 'z') o a dígitos. Aprender el modelo es aprender estas regularidades a partir de los datos.

Las ventanas $\{(X_i, Y_i)\}$ son solapadas: ventanas consecutivas comparten $B - 1$ tokens, por lo que las observaciones *no* son iid. En consecuencia, los errores estándar reportados por batch deben leerse como *descriptivos*: dan una idea del orden de magnitud de la fluctuación observada entre minibatches, pero no son intervalos de confianza válidos en sentido inferencial (un intervalo de confianza clásico requeriría observaciones iid).

Para la evaluación, la sucesión tokenizada se divide en tres partes contiguas con `train_val_test_split`. El primer 80% de los tokens forma el conjunto de *entrenamiento*, sobre el cual se ajustan los parámetros w por descenso por gradiente; el siguiente 10% forma el conjunto de *validación*, que monitorea el progreso del entrenamiento (típicamente, detecta sobreajuste comparando pérdida de entrenamiento con pérdida de validación); el último 10% forma el conjunto de *test*, reservado para la evaluación final. La división es contigua y no se barajan los tokens antes de dividir: el test corresponde literalmente a la última porción del texto. Esto preserva la coherencia local del corpus dentro de cada split, a costa de no garantizar que train, validation y test sean iid.

4 Red neuronal entrenable

De manera abstracta, una *red neuronal feedforward* es una función parametrizada $f_w : \mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^{d_{\text{out}}}$ obtenida componiendo capas alternadas de transformaciones afines $x \mapsto Wx + b$ (con W una matriz y b un vector aprendibles) y funciones de activación no lineales (ReLU, GELU, SiLU, etc.) aplicadas coordenada a coordenada [6, Cap. 2]. Los parámetros w son la concatenación de los pesos W y sesgos b de cada capa, y se ajustan minimizando una pérdida sobre datos. En modelado de lenguaje, las entradas son tokens (no vectores reales), pero la primera capa los convierte en vectores mediante una tabla de embeddings, y a partir de ahí la red opera sobre $\mathbb{R}^d$.

Definición 4 (Red neuronal entrenable). Sea $w \in \mathbb{R}^n$ el vector de parámetros (embeddings, matrices lineales, pesos de atención, ganancias de normalización, MLPs). La red neuronal entrenable usada aquí es la función parametrizada

$$f_w : \{0, \dots, |\mathcal{V}| - 1\}^B \rightarrow \mathbb{R}^{B \times |\mathcal{V}|},$$

que recibe una sucesión de B tokens y devuelve *logits* indexados por posición t y por símbolo j del vocabulario.

El bigrama de §9 es un modelo paramétrico equivalente a una tabla de logits y *no* se llama aquí red neuronal profunda; el Transformer (`transformer.py`) y el modelo LLaMA-style (`llama.py`) sí lo son, en el sentido de Petersen-Zech [6, Cap. 2 y Cap. 17.3].

5 Modelo probabilístico autoregresivo

Un modelo de lenguaje autoregresivo es una familia de medidas de probabilidad discretas $\{p_w\}$ sobre sucesiones finitas de

tokens que factoriza

$$p_w(x_0, \dots, x_{T-1}) = \prod_{t=0}^{T-1} p_w(x_t \mid x_{<t}). \quad (3)$$

Para una ventana $X = (x_0, \dots, x_{B-1})$ y una posición t , denotamos por $f_w(X)_{t,:} \in \mathbb{R}^{|\mathcal{V}|}$ la t -ésima fila de la matriz de logits que devuelve la red: es decir, los logits que el modelo asigna a cada candidato del vocabulario para la posición siguiente, dado el contexto X . Los logits son números reales sin normalizar, no probabilidades.

Para obtener una distribución de probabilidad sobre $\mathcal{V}$, se aplica la *función softmax* $\sigma : \mathbb{R}^k \rightarrow \Delta^{k-1}$, definida por

$$\sigma(z)_j = \frac{e^{z_j}}{\sum_{i=1}^k e^{z_i}}, \quad j = 1, \dots, k.$$

Por construcción $\sigma(z)_j > 0$ y $\sum_j \sigma(z)_j = 1$, así que $\sigma(z)$ pertenece al simplex de probabilidad Δ^{k-1} . La probabilidad condicional inducida por el modelo es

$$p_w(x_{t+1} = j \mid X) = \sigma(f_w(X)_{t,:})_j. \quad (4)$$

La distribución resultante es una *distribución categórica* sobre $\mathcal{V}$: una asignación de probabilidades a cada elemento del vocabulario, formalmente análoga a tirar un dado de $|\mathcal{V}|$ caras con probabilidades posiblemente desiguales. Por ejemplo, con un mini-vocabulario $\mathcal{V} = \{ 'a', 'b', 'c' \}$ y logits $(2.0, 0.5, 0.1)$, softmax devuelve aproximadamente $(0.74, 0.17, 0.09)$, que se interpreta así: dado el contexto actual, el modelo asigna probabilidad 74% a que el siguiente carácter sea 'a', 17% a 'b' y 9% a 'c'.

El modelo, pues, no predice texto directamente: predice una distribución categórica por posición. La generación se realiza muestreando del softmax del último logit (método `generate` en cada modelo).

6 Función de pérdida

Definición 5 (Pérdida token a token). Para una ventana (X, Y) y parámetros w , la *entropía cruzada* por posición es

$$\ell(f_w(X), Y) = \frac{1}{B} \sum_{t=1}^B -\log \text{softmax}(f_w(X)_{t,:})_{Y_t}.$$

Esta cantidad es el *negative log-likelihood* bajo el modelo (3); minimizarla equivale a maximizar la verosimilitud condicional del token correcto.

Calin [7] desarrolla la entropía cruzada como costo natural al comparar la distribución empírica con la del modelo, vía la divergencia de Kullback-Leibler. Wegner [8] deriva, para salida softmax, la identidad

$$\nabla_z [-\log \text{softmax}(z)_y] = \text{softmax}(z) - e_y,$$

con e_y la indicatriz one-hot. En el código, esta identidad se evalúa implícitamente vía `F.cross_entropy` en cada modelo y propaga gradientes mediante `loss.backward()`.

7 Riesgo poblacional y riesgo empírico

Definición 6 (Riesgo poblacional, Petersen-Zech [6, Def. 14.2]). Sea $\mathcal{D}$ una distribución desconocida sobre pares (X, Y) . El *riesgo poblacional* de f_w es

$$R(w) = \mathbb{E}_{(X,Y) \sim \mathcal{D}} [\ell(f_w(X), Y)].$$

Definición 7 (Riesgo empírico, Petersen-Zech [6, Def. 14.4]). Dado un dataset $S = \{(X_i, Y_i)\}_{i=1}^m$,

$$\hat{R}_S(w) = \frac{1}{m} \sum_{i=1}^m \ell(f_w(X_i), Y_i).$$

La distribución $\mathcal{D}$ no es observable; el dataset S sí, y se obtiene del corpus por (2). Aprender significa, en el sentido de Petersen-Zech [6, Def. 14.5], buscar w que reduzca $\hat{R}_S(w)$, no $R(w)$ directamente. La generalización se evalúa comparando pérdida de train y test. La diferencia es central: Wegner muestra que las redes tienen amplia capacidad representacional, pero la efectividad estadística depende también de los datos finitos y de la optimización.

8 Optimización

El script `train.py` utiliza AdamW [? ?], una variante de descenso estocástico por gradiente con momentos adaptativos y decaimiento de pesos desacoplado. Sea $g_k = \nabla_w \hat{R}_{B_k}(w_k)$ el gradiente sobre el minibatch B_k de tamaño $M = 32$; la actualización extiende la de Petersen-Zech [6, §10.4] con un término de decaimiento desacoplado:

$$u_{k+1} = \beta_1 u_k + (1 - \beta_1) g_k, \quad (5)$$

$$s_{k+1} = \gamma_1 s_k + (1 - \gamma_1) g_k \odot g_k, \quad (6)$$

$$w_{k+1} = w_k - \alpha (\hat{u}_{k+1} \odot (\sqrt{\hat{s}_{k+1}} + \varepsilon) + \lambda w_k), \quad (7)$$

donde $\hat{u}_k = u_k / (1 - \beta_1^k)$ y $\hat{s}_k = s_k / (1 - \gamma_1^k)$, $\beta_1, \gamma_1 \in (0, 1)$ son los factores de decaimiento del primer y segundo momento (defaults 0.9 y 0.999), $\varepsilon = 10^{-8}$ estabiliza la división, $\odot$ y $\oslash$ son producto y cociente coordenada a coordenada, $\alpha = 3 \cdot 10^{-4}$ es la tasa global y $\lambda = 0.01$ es el coeficiente de *weight decay* desacoplado de [?] (default de PyTorch). Una iteración de entrenamiento ejecuta `optimizer.zero_grad`, `loss.backward()` y `optimizer.step()`. El algoritmo minimiza el riesgo empírico de entropía cruzada *minibatch a minibatch*; Petersen-Zech enmarca esto como minimización estocástica del objetivo $F = \hat{R}_S$ [6, §10.2], y Calin como estimación adaptativa de momentos del gradiente [7].

9 Modelos implementados

Bigrama. `BigramLanguageModel` es una *cadena de Markov de orden uno*. Una cadena de Markov es un proceso estocástico $(x_0, x_1, x_2, \dots)$ sobre un espacio de estados (aquí, el vocabulario $\mathcal{V}$) que satisface la propiedad de Markov: la

distribución del siguiente estado depende solo del actual, no de todo el pasado. Formalmente, de orden uno significa

$$p(x_{t+1} | x_0, \dots, x_t) = p(x_{t+1} | x_t).$$

El bigrama aprende una matriz $A \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$ realizada como `nn.Embedding(vocab_size, vocab_size)` (cada fila es el vector de logits asociado a un estado de partida) y modela

$$p_w(x_{t+1} = j | x_t = i) = \sigma(A_{i,:})_j.$$

La fila $A_{i,:}$ son los logits de la distribución condicional de x_{t+1} dado $x_t = i$. El modelo ignora todo el contexto excepto el último carácter; sirve aquí como referencia conceptual del régimen sin memoria larga.

Transformer causal. `TinyTransformerLanguageModel` [9] es una red neuronal decoder-only que procesa la sucesión de tokens en tres etapas. Primero, cada token $id\ x_t \in \{0, \dots, |\mathcal{V}| - 1\}$ se transforma en un vector denso $e_t \in \mathbb{R}^d$ (con $d = 64$) consultando una tabla de embeddings aprendible (`nn.Embedding(|\mathcal{V}|, d)`); a este se le suma un *embedding posicional* aprendible que codifica la posición absoluta t dentro del bloque. Segundo, la sucesión de embeddings $H \in \mathbb{R}^{B \times d}$ atraviesa $L = 2$ bloques pre-norm, cada uno de la forma

$$H \leftarrow H + \text{Attn}(\text{LayerNorm}(H)), \quad H \leftarrow H + \text{FFN}(\text{LayerNorm}(H)),$$

donde las sumas (*conexiones residuales*) facilitan la propagación de gradientes; `LayerNorm` normaliza cada vector token a media cero y varianza unitaria; y `FFN` es un MLP posición-a-posición con activación GELU. Tercero, la salida pasa por una norma final y una proyección lineal a logits, `lm_head: \mathbb{R}^d \rightarrow \mathbb{R}^{|\mathcal{V}|}, que produce  $f_w(X)$ .`

El componente central es la *atención causal multi-cabeza*. Para cada cabeza se calculan tres matrices a partir de H :

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V, \quad (8)$$

con $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_h}$ aprendibles. La salida de la cabeza es

$$\text{Attn}(H) = \sigma \left(\frac{QK^\top}{\sqrt{d_h}} + M_c \right) V. \quad (9)$$

Intuitivamente, cada fila q_t de Q representa una *consulta* (qué información necesita la posición t); cada fila k_s de K representa una *clave* (qué información ofrece la posición s); el producto interno $q_t \cdot k_s$ mide compatibilidad. El softmax convierte estas compatibilidades en pesos de mezcla que se aplican sobre los *valores* V . La división por $\sqrt{d_h}$ estabiliza los gradientes al evitar que los argumentos del softmax crezcan con la dimensión. La *máscara causal* M_c de `causal_mask.py` es triangular inferior: $M_{c,ts} = 0$ si $s \leq t$ y $-\infty$ si $s > t$, garantizando que la posición t solo atienda a posiciones $s \leq t$ (el $-\infty$ en el argumento del softmax produce peso cero). `MultiHeadCausalSelfAttention` ejecuta $h = 4$ cabezas independientes en paralelo, cada una con dimensión $d_h = d/h = 16$, concatena sus salidas y aplica una proyección lineal final. Esto permite que cabezas distintas se especialicen en patrones diferentes (similitud léxica, posición relativa, dependencias estructurales).

LLaMA-style. `LlamaStyleLanguageModel` mantiene la arquitectura general (decoder-only, atención causal, conexiones residuales, pre-norm) y la misma tarea y pérdida, pero sustituye tres piezas internas siguiendo [10].

RoPE: embeddings rotativos. En lugar de sumar un embedding posicional aprendido al embedding de token (como en el Transformer estándar), RoPE [11] codifica la posición *rotando* las queries y keys por un ángulo dependiente de la posición absoluta. Las d_h coordenadas de cada query/key se agrupan en pares (x_{2i}, x_{2i+1}) , y para la posición p se aplica una rotación 2D:

$$\begin{pmatrix} x'_{2i} \\ x'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos p\theta_i & -\sin p\theta_i \\ \sin p\theta_i & \cos p\theta_i \end{pmatrix} \begin{pmatrix} x_{2i} \\ x_{2i+1} \end{pmatrix},$$

con frecuencias $\theta_i = 10000^{-2i/d_h}$. La propiedad clave es que el producto interno $\langle q_t, k_s \rangle$ tras aplicar RoPE depende solo de la diferencia $t - s$ (posición relativa), no de posiciones absolutas. Implementado en `rope.py`.

RMSNorm: normalización simplificada. La `LayerNorm` estándar normaliza cada vector restando la media y dividiendo por la desviación estándar; RMSNorm [12] suprime el paso de centrado:

$$\text{RMSNorm}(x) = g \odot \frac{x}{\sqrt{d^{-1} \sum_j x_j^2 + \varepsilon}},$$

con $g \in \mathbb{R}^d$ una ganancia aprendible. Argumentos empíricos sugieren que la información útil del centrado es marginal y RMSNorm resulta computacionalmente más barata. Definida en `rmsnorm.py`.

SwiGLU: feedforward con compuerta. El MLP posición-a-posición estándar es $\text{FFN}(x) = W_2 \text{GELU}(W_1 x)$. SwiGLU [13] introduce un mecanismo de compuerta multiplicativa:

$$\text{SwiGLU}(x) = W_d (\text{SiLU}(W_g x) \odot W_u x),$$

donde $\text{SiLU}(x) = x \cdot \sigma_{\log\text{stica}}(x)$. La rama $W_u x$ propaga información mientras la rama $\text{SiLU}(W_g x)$ actúa como compuerta dinámica que modula coordenada a coordenada qué pasa. Empíricamente mejora el desempeño respecto a un MLP simple. Definido en `swiglu.py`, con proyecciones sin sesgo. Así, Transformer y LLaMA-style comparan dos sesgos inductivos para la misma distribución autoregresiva.

10 Métricas

Diversidad del corpus. Un k -grama es una subcadena contigua $g = (c_i, \dots, c_{i+k-1})$. Sea $\hat{p}_k(g) = \#\{i : (c_i, \dots, c_{i+k-1}) = g\} / (N - k + 1)$ la frecuencia empírica de g . El módulo `corpus_stats.py` computa

$$H_k = - \sum_g \hat{p}_k(g) \log \hat{p}_k(g) \quad (\text{en nats}),$$

$$D_n = \frac{\#\{n\text{-gramas únicos}\}}{\#\{n\text{-gramas totales}\}},$$

$$\rho_{\text{gzip}} = \frac{|\text{gzip}(s)|}{|s|}.$$

Cada una captura una faceta distinta del corpus. H_k mide la *incertidumbre estadística promedio* de los k -gramas: H_k alto significa que muchos k -gramas distintos aparecen con probabilidades comparables; H_k bajo significa que unos pocos k -gramas dominan la distribución empírica. D_n mide la *fracción de n -gramas únicos*: cercana a 1 cuando casi todos los n -gramas son distintos (poca repetición exacta de bloques cortos), cercana a 0 cuando hay mucha repetición. ρ_{gzip} aproxima la *redundancia compresible* detectable por LZ77: valores cercanos a 1 indican baja redundancia (texto difícil de comprimir); valores cercanos a 0 indican mucha redundancia. ρ_{gzip} se relaciona con la razón de compresión usada por Chang et al. [3] en el límite de no-redundancia.

Las tres son *medidas distribucionales* del texto: capturan propiedades estadísticas observables sobre las frecuencias de subcadenas. Lo que *no* capturan: coherencia semántica (si el texto tiene sentido), factualidad (si lo que afirma es verdadero), ni syntheticity (cuán típico es el texto bajo un modelo de lenguaje entrenado).

Evaluación. Sobre el split de test se reportan cuatro cantidades. La *pérdida de test* $\hat{R}_{\text{test}}$ es el riesgo empírico de entropía cruzada (Definición 4 arriba ya formalizó la pérdida; aquí se evalúa sobre el conjunto de test): pérdida más baja indica mayor probabilidad asignada al token correcto. La *pérdida de entrenamiento* $\hat{R}_{\text{train}}$ es la misma cantidad sobre el split de entrenamiento. La *perplejidad* $\text{PPL} = \exp(\hat{R}_{\text{test}})$ es una transformación monotónica de la pérdida y, por tanto, ordena los modelos exactamente igual; aun así, se reporta porque su interpretación es más intuitiva: si $\text{PPL} = k$, el modelo se comporta, en promedio, como si en cada posición tuviera que elegir uniformemente entre k tokens. Esto permite contrastar resultados con la literatura, que históricamente usa perplejidad en lugar de pérdida. La *brecha de generalización* $\text{gap} = \hat{R}_{\text{test}} - \hat{R}_{\text{train}}$ mide la diferencia entre desempeño dentro y fuera de entrenamiento; gap cercano a cero (o negativo) sugiere ajuste justo o subajuste, y gap positivo grande sugiere sobreajuste.

11 Protocolo experimental

Se comparan dos corpus de 100 000 caracteres, *Medium* (más homogéneo) y *High* (mezcla de fuentes), con el mismo presupuesto de entrenamiento. El script `run_effective_tokens.py` ejecuta el pipeline corpus por corpus y modelo por modelo y agrega resultados con `summarize_results.py`. La Tabla 1 resume los hiperparámetros comunes.

12 Resultados

Las columnas del cuadro tienen el siguiente significado. *Corpus* es la etiqueta del corpus; *Medium* y *High* se eligen por su nivel relativo de diversidad, no por su tamaño (ambos

Table 1: Presupuesto común de entrenamiento.

Cantidad	Valor
Tokenización	nivel carácter
Contexto B	64
Batch size M	32
Dimensión d	64
Capas/cabezas	2 / 4
Optimizador	AdamW
Learning rate α	$3 \cdot 10^{-4}$
Pasos de entrenamiento	2 000
Runs por configuración	1

Table 2: Diversidad empírica de los dos corpus.

Corpus	Vocab	H_3	Dist.-4	Gzip
Medium	94	7.4058	0.0378	0.3579
High	102	7.5880	0.0592	0.4066

tienen $N = 1\,000\,000$ caracteres). *Vocab* es $|\mathcal{V}|$, el tamaño del vocabulario a nivel carácter, esto es, el número de caracteres distintos que aparecen en el corpus. H_3 es la entropía empírica de los 3-gramas en nats; reportar específicamente H_3 (y no H_1 , H_2 , H_4 o más altos) responde a un compromiso: H_1 ignora todo el orden y H_k con k grande se vuelve ralo al tamaño finito del corpus (muchos k -gramas observados solo una vez), lo que sesga el estimador plug-in. *Dist.-4* es la fracción de 4-gramas únicos sobre el total de 4-gramas; valores cercanos a 1 indican poca repetición exacta de bloques de cuatro caracteres. *Gzip* es ρ_{gzip} , razón entre tamaño comprimido y tamaño original; valores cercanos a 1 indican baja redundancia compresible. En conjunto, las cuatro métricas dan una imagen multifacética: tamaño del alfabeto, incertidumbre estadística local, repetición exacta de bloques cortos y compresibilidad global.

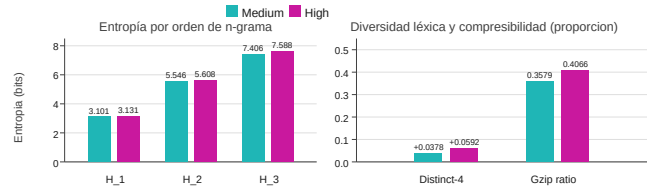


Figure 1: El corpus *High* domina a *Medium* en diversidad local: mayor entropía de n -gramas, mayor distinct-4 y mayor razón gzip.

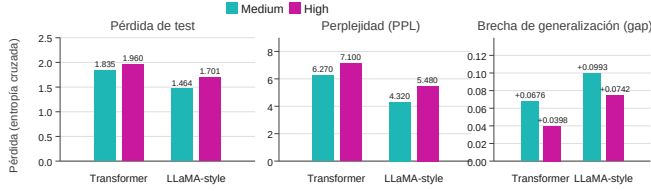
El efecto es consistente pero moderado a esta escala. En Transformer, pasar de *Medium* a *High* aumenta la pérdida de test en 0.124 y la perplejidad en aproximadamente 13%. En LLaMA-style, la pérdida aumenta 0.237 y la perplejidad en 27%. Por tanto,

$$D(\text{High}) > D(\text{Medium}) \quad \text{pero} \quad \hat{R}_{\text{test}}(\text{High}) > \hat{R}_{\text{test}}(\text{Medium}).$$

La pregunta (1) se responde en sentido negativo para el riesgo absoluto. Sin embargo, la brecha de generalización exhibe el patrón inverso: $\text{gap}(\text{High}) < \text{gap}(\text{Medium})$ en ambas arquitecturas (0.040 vs. 0.068 en Transformer; 0.074 vs. 0.099 en LLaMA-style). Con 10^6 caracteres, la mayor diversidad reduce el sobreajuste, aunque sin reducir la pérdida absoluta.

Table 3: Evaluación en test. $PPL = \exp(\hat{R}_{\text{test}})$, $\text{gap} = \hat{R}_{\text{test}} - \hat{R}_{\text{train}}$.

Modelo	Corpus	Loss	PPL	Gap
Transformer	Medium	1.8355	6.27	0.0676
Transformer	High	1.9597	7.10	0.0398
LLaMA-style	Medium	1.4638	4.32	0.0993
LLaMA-style	High	1.7010	5.48	0.0742

**Figure 2:** A igual presupuesto, *High* empeora pérdida, perplejidad y brecha en ambas arquitecturas.

13 Discusión

La interpretación rigurosa no es que la diversidad perjudique universalmente. El resultado muestra que, en modelos pequeños entrenados desde cero, mayor diversidad local puede aumentar la incertidumbre condicional efectiva del problema más rápido de lo que la red puede aprovecharla con 5 000 pasos de AdamW. La caída del riesgo se descompone en términos de aproximación, optimización y generalización [6, Cap. 14]; aquí los tres factores empujan a favor de *Medium*. El resultado no contradice los tokens efectivos de Chang et al. [3]; los delimita: un token es efectivo solo relativamente a una arquitectura, un tokenizador, un optimizador y una escala.

Amenazas a la validez. *High* puede mezclar diversidad con cambio de dominio entre fragmentos; hay una sola *semilla* por configuración (por semilla se entiende el estado inicial del generador de números pseudoaleatorios del experimento; controla la inicialización de los pesos del modelo y el orden en que se muestrean los minibatches durante el entrenamiento; con una sola semilla, las pérdidas reportadas son una realización del experimento, no un promedio sobre realizaciones independientes); la tokenización a carácter ignora estructura subpalabra; la escala (10^6 tokens) es órdenes de magnitud menor que la de LLMs reales. Controles naturales son repetir el experimento con varias semillas, construir corpus con diversidad controlada dentro de una sola fuente, variar el presupuesto de entrenamiento y comparar con tokenización subpalabra.

14 Conclusión

El estudio formaliza los objetos matemáticos que ejecuta *nicolas-lm* (red neuronal entrenable, modelo autoregresivo, riesgo poblacional y empírico, AdamW, atención causal, RMSNorm, RoPE, SwiGLU) y evalúa una pregunta concreta sobre tokens efectivos: si mayor diversidad textual empírica implica mayor efectividad. Con $N = 10^6$ caracteres y 5 000

pasos de AdamW, el resultado es matizado: el corpus más diverso produce pérdida y perplejidad absolutas más altas en ambas arquitecturas, lo que responde negativamente la pregunta principal; pero a la vez exhibe una brecha de generalización menor, sugiriendo que la diversidad sí modera el sobreajuste a esta escala. La efectividad de los tokens no se reduce a una sola métrica: en riesgo absoluto la homogeneidad gana; en generalización relativa, la diversidad ayuda.

References

- [1] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. URL <https://arxiv.org/abs/2001.08361>.
- [2] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022. URL <https://arxiv.org/abs/2203.15556>.
- [3] Ernie Chang, Matteo Paltenghi, Yang Li, Pin-Jie Lin, Changsheng Zhao, Patrick Huber, Zechun Liu, Rastislav Rabatin, Yangyang Shi, and Vikas Chandra. Scaling parameter-constrained language models with quality data. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 80–97, Miami, Florida, US, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-industry.8. URL <https://aclanthology.org/2024.emnlp-industry.8/>.
- [4] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 1715–1725, 2016. doi: 10.18653/v1/P16-1162. URL <https://aclanthology.org/P16-1162/>.
- [5] Taku Kudo and John Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, 2018. doi: 10.18653/v1/D18-2012. URL <https://aclanthology.org/D18-2012/>.
- [6] Philipp Petersen and Jakob Zech. Mathematical theory of deep learning, 2026. URL <https://arxiv.org/abs/2407.18384>.
- [7] Ovidiu Calin. *Deep Learning Architectures: A Mathematical Approach*. Springer Series in the Data Sciences. Springer, Cham, 2020. ISBN 978-3-030-36721-3. doi: 10.1007/978-3-030-36721-3.
- [8] Sven A. Wegner. *Mathematical Introduction to Data Science*. Springer, Berlin, Heidelberg, 2024. ISBN 978-3-662-69426-8. doi: 10.1007/978-3-662-69426-8.
- [9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia

- Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL <https://papers.nips.cc/paper/7181-attention-is-all-you-need>.
- [10] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothee Lacroix, Baptiste Roziere, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023. URL <https://arxiv.org/abs/2302.13971>.
- [11] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021. URL <https://arxiv.org/abs/2104.09864>.
- [12] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://papers.nips.cc/paper/9403-root-mean-square-layer-normalization>.
- [13] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020. URL <https://arxiv.org/abs/2002.05202>.